

```
# %%  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score  
  
  
import tensorflow as tf  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense  
  
  
# %%  
  
df = pd.read_csv('HousingData.csv')  
  
  
# %%  
  
df.head()  
  
  
# %%  
  
df.shape  
  
  
# %%  
  
df.info()  
  
  
# %%  
  
df.describe()
```

```
# %%
```

```
data.isnull().sum()
```

```
# %%
```

```
df.fillna(df.mean(), inplace=True)
```

```
# %%
```

```
df.isnull().sum()
```

```
# %%
```

```
X = df.drop("MEDV", axis=1)
```

```
y = df["MEDV"]
```

```
# %%
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# %%
```

```
X_train.shape
```

```
# %%
```

```
X_test.shape
```

```
# %%
```

```
scaler = StandardScaler()
```

```
# %%
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# %%  
  
model = Sequential()  
  
# %%  
  
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))  
model.add(Dense(32, activation='relu'))  
model.add(Dense(16, activation='relu'))  
model.add(Dense(1))  
  
# %%  
  
model.compile(optimizer='adam', loss='mse', metrics=['mae'])  
  
# %%  
  
model.summary()  
  
# %% [markdown]  
  
### history = model.fit(X_train, y_train, validation_split=0.2, epochs=100,  
batch_size=16, verbose=1)  
  
# %%  
  
y_pred = model.predict(X_test)  
  
# %%  
  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)
```

```
# %%
```

```
print("MSE : ",mse)
```

```
print("RMSE : ",rmse)
```

```
print("MAE : ",mae)
```

```
print("R2 score : ",r2)
```

```
# %%
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(y_test.values[:30], label='Actual Price')
```

```
plt.plot(y_pred[:30], label="Predicted Price")
```

```
plt.xlabel("House samples")
```

```
plt.ylabel("Price")
```

```
plt.title("Actual vs Predicted House Prices")
```

```
plt.legend()
```

```
plt.show()
```

```
# %%
```

```
# %%
```

```
=====
```

```
# %%
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# %%
df = pd.read_csv("letter-recognition.data", header=None)
```

```
# %%
df.head()
```

```
# %%
df.shape
```

```
# %%
df.info()
```

```
# %%
X = df.iloc[:, 1:]
y = df.iloc[:, 0]
```

```
# %%
X.shape
```

```
# %%
```

```
y.shape
```

```
# %%
```

```
encoder = LabelEncoder()
```

```
# %%
```

```
y = encoder.fit_transform(y)
```

```
# %%
```

```
scaler = StandardScaler()
```

```
# %%
```

```
X = scaler.fit_transform(X)
```

```
# %%
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# %%
```

```
model = Sequential()
```

```
# %%
```

```
model.add(Dense(128, activation='relu', input_shape=(16,)))
```

```
model.add(Dense(64, activation='relu'))
```

```
model.add(Dense(32, activation='relu'))
```

```
#output layer = 26 classes
```

```
model.add(Dense(26, activation='softmax'))
```

```
# %%  
  
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])  
  
# %%  
  
model.summary()  
  
# %%  
  
history = model.fit(X_train, y_train, validation_split=0.2, epochs=30, batch_size=32,  
verbose=1)  
  
# %%  
  
y_pred_prob = model.predict(X_test)  
y_pred = np.argmax(y_pred_prob, axis=1)  
  
# %%  
  
accuracy = accuracy_score(y_test, y_pred)  
  
accuracy  
  
# %%  
  
print(classification_report(y_test, y_pred))  
  
# %%  
  
plt.figure(figsize=(8,5))  
  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title("Training vs Validation Accuracy")  
plt.xlabel("Epochs")  
plt.ylabel("Accuracy")
```

```
plt.legend()
```

```
plt.show()
```

```
# %%
```

```
=====
```

```
# %%
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D
```

```
# %%
```

```
train_df = pd.read_csv("fashion-mnist_train.csv")
```

```
test_df = pd.read_csv("fashion-mnist_test.csv")
```

```
# %%
```

```
train_df.shape
```

```
# %%
```

```
test_df.shape
```



```
# %%
```

```
X_train = train_df.iloc[:, 1:].values
```

```
y_train = train_df.iloc[:, 0].values
```

```
X_test = test_df.iloc[:, 1:].values
```

```
y_test = test_df.iloc[:, 0].values
```

```
# %%
```

```
#Normalization
```

```
X_train = X_train / 255.0
```

```
X_test = X_test / 255.0
```

```
# %%
```

```
#Reshape for CNN
```

```
X_train = X_train.reshape(-1, 28,28,1)
```

```
X_test = X_test.reshape(-1,28,28,1)
```

```
# %%
```

```
#Build CNN model
```

```
model = Sequential()
```

```
# %%
```

```
model.add(
```

```
    Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(28,28,1))
```

```
)
```

```
# %%
```

```
model.add(
```

```
    MaxPooling2D(pool_size=(2,2))
)

# %%

model.add(Flatten())

# %%

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.2))

# %%

model.add(Dense(10, activation='softmax'))

# %%

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# %%

model.summary()

# %%

history = model.fit(X_train, y_train, validation_split=0.2, epochs=10, batch_size=32,
verbose=1)

# %%

y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)

# %%
```

```
accuracy = accuracy_score(y_test, y_pred)

accuracy

# %%

print(classification_report(y_test, y_pred))

# %%

class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker',
               'Bag', 'Ankle Boot']

index = 0

plt.figure(figsize=(4,4))

plt.imshow(X_test[index].reshape(28,28))

plt.title("Actual: "+class_names[y_test[index]] + "\nPredicted: " +
          class_names[y_pred[index]])

plt.show()

# %%


---



---



# %%

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error
```

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, SimpleRNN


# %%

df = pd.read_csv("goog.csv")


# %%

df.head()


# %%

df.shape


# %%

df.info()


# %%

data = df[['Close']].values


# %%

data.shape


# %%

scaler = MinMaxScaler()


# %%

data_scaled = scaler.fit_transform(data)
```

```
# %%  
  
#create sequence  
  
#using previous 10 days  
  
X = []  
y = []  
  
  
time_steps = 10  
  
for i in range(time_steps, len(data_scaled)):  
    X.append(data_scaled[i-time_steps : i, 0])  
    y.append(data_scaled[i, 0])  
  
  
X = np.array(X)  
y = np.array(y)  
  
  
# %%  
  
#reshape for RNN  
  
X = X.reshape(X.shape[0], X.shape[1], 1)  
  
  
print(X.shape)  
print(y.shape)  
  
  
# %%  
  
split = int(0.8 * len(X))  
  
  
X_train = X[:split]  
X_test = X[split:]
```

```
y_train = y[:split]
```

```
y_test = y[split:]
```

```
# %%
```

```
X_train.shape
```

```
# %%
```

```
#Build RNN model
```

```
model = Sequential()
```

```
# %%
```

```
model.add(
```

```
    SimpleRNN(units=100, activation='tanh', input_shape=(X_train.shape[1], 1))
```

```
)
```

```
# %%
```

```
model.add(Dense(1))
```

```
# %%
```

```
model.compile(optimizer='adam', loss='mse')
```

```
# %%
```

```
model.summary()
```

```
# %%
```

```
history = model.fit(X_train, y_train, epochs=100, batch_size=8, verbose=1)
```

```
# %%  
  
y_pred = model.predict(X_test)  
  
# %%  
  
y_test = scaler.inverse_transform(y_test.reshape(-1, 1))  
  
# %%  
  
y_pred = scaler.inverse_transform(y_pred)  
  
# %%  
  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
  
rmse  
  
# %%  
  
plt.figure(figsize=(10,5))  
plt.plot(y_test, label='Actual Price')  
plt.plot(y_pred, label='Predicted Price')  
  
plt.title("Google Stock Price Prediction")  
plt.xlabel("Days")  
plt.ylabel("Stock Price")  
  
plt.legend()  
plt.show()  
  
# %%
```

